

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quién ganará? Greedys vs Dinámico vs Bruto.»

Martín Vicente Mella Chávez

22 de junio de 2026

22:17

1. Entorno experimental

Los experimentos fueron realizados en un computador personal Legion 5 15ITH6 utilizando sistema operativo Linux (WSL sobre Windows mediante Ubuntu 22.04 LTS), con procesador 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz (2.69 GHz) y 8 GB de memoria RAM. El código fue compilado utilizando g++ con estándar C++.

El programa mide automáticamente el tiempo de ejecución mediante la librería chrono y el uso de memoria mediante getrusage. La ejecución de pruebas se automatizó mediante un Makefile, generando resultados reproducibles.

Los datasets utilizados corresponden a instancias sintéticas del problema, variando el número de animes n en valores: 5, 10, 15, 20, 30, 50, 80, 120 y 200. Para cada tamaño se generaron múltiples casos de prueba, permitiendo calcular promedios y analizar el comportamiento de los algoritmos.

Los resultados se almacenan automáticamente en archivos dentro de `data/measurements/`, y posteriormente se procesan mediante scripts en Python para generar gráficos en `data/plots/`.

2. Resultados y Análisis

2.1. Complejidad de los algoritmos

El algoritmo de fuerza bruta explora exhaustivamente todas las combinaciones posibles de prefijos para cada anime. Si un anime posee q_i capítulos, entonces existen $q_i + 1$ posibles decisiones para dicho anime. Como consecuencia, el número total de combinaciones crece exponencialmente respecto al número de animes.

La complejidad temporal del algoritmo de fuerza bruta es exponencial, lo que limita su aplicación únicamente a instancias pequeñas. El consumo de memoria se encuentra asociado principalmente a la profundidad de la recursión utilizada por el backtracking.

La primera heurística greedy construye una lista con todos los prefijos posibles y los ordena utilizando una relación entre satisfacción obtenida y costo de recursos consumidos. El criterio local utilizado consiste en maximizar la relación entre satisfacción obtenida y recursos consumidos, priorizando aquellos prefijos que entregan una mayor cantidad de satisfacción por unidad de tiempo y energía.

La segunda heurística greedy selecciona para cada anime el prefijo que presenta la mejor relación local entre satisfacción y recursos utilizados. El criterio utilizado consiste en escoger la mejor alternativa individual para cada anime antes de construir la solución global.

La solución mediante programación dinámica modela el problema utilizando estados asociados al número de animes considerados, el tiempo disponible y la energía restante. Esto permite garantizar la obtención de la solución óptima para las instancias que pueden resolverse dentro de los límites de memoria y tiempo.

La Tabla 1 resume las complejidades temporales teóricas de los algoritmos implementados.

Algoritmo	Complejidad temporal
Fuerza bruta	Exponencial
Greedy 1	$O(NQ \log(NQ))$
Greedy 2	$O(NQ)$
Programación dinámica	$O(NME)$

Cuadro 1: Complejidad temporal de los algoritmos implementados.

Los resultados obtenidos se presentan mediante gráficos generados automáticamente a partir de los datos recolectados durante la ejecución de los distintos algoritmos sobre las instancias generadas.

2.2. Análisis del tiempo de ejecución



Figura 1: Comparación de tiempos de ejecución

En la Figura 1 se observa el comportamiento temporal de los algoritmos implementados a medida que aumenta el tamaño de las instancias. El algoritmo de fuerza bruta presenta un crecimiento acelera-

do en sus tiempos de ejecución, dejando de ser práctico para valores superiores a $n = 25$. Este comportamiento resulta consistente con la complejidad exponencial del algoritmo, ya que explora exhaustivamente todas las combinaciones posibles de prefijos.

Los algoritmos Greedy 1 y Greedy 2 presentan tiempos de ejecución considerablemente menores. Ambos mantienen tiempos prácticamente constantes incluso para las instancias más grandes consideradas durante los experimentos, debido a que sus decisiones se basan en criterios locales y requieren únicamente operaciones de ordenamiento y recorridos lineales.

Por otro lado, la programación dinámica presenta un crecimiento considerable del tiempo de ejecución a medida que aumenta el tamaño de las instancias. Si bien el costo computacional aumenta para instancias grandes, el algoritmo continúa siendo mucho más eficiente que la fuerza bruta y mantiene la capacidad de obtener soluciones óptimas.

En consecuencia, la programación dinámica ofrece un equilibrio adecuado entre tiempo de ejecución y calidad de solución, constituyéndose como una alternativa viable para resolver el problema en la práctica.

2.3. Calidad de las soluciones

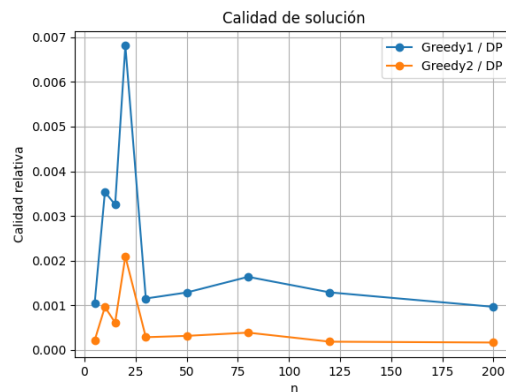


Figura 2: Calidad de solución de algoritmos greedy respecto a programación dinámica

La Figura 2 compara la calidad de las soluciones obtenidas por los algoritmos greedy respecto a la programación dinámica, la cual se considera como referencia óptima.

Se observa que los algoritmos greedy no siempre alcanzan la solución óptima. En diversas instancias, particularmente en tamaños grandes, la diferencia respecto a la programación dinámica se vuelve significativa. Esto ocurre porque las heurísticas realizan decisiones locales que pueden impedir alcanzar la mejor solución global.

El algoritmo Greedy 2 presenta un comportamiento ligeramente superior al Greedy 1 en varios casos, debido a que considera la mejor opción individual para cada anime antes de realizar la selección global. Sin embargo, ninguno de los enfoques greedy garantiza optimalidad.

La programación dinámica, por su parte, coincide con la fuerza bruta, esto permitió validar experi-

mentalmente la correctitud de la implementación de programación dinámica utilizando las instancias pequeñas donde ambos algoritmos pueden ejecutarse.

2.4. Uso de memoria

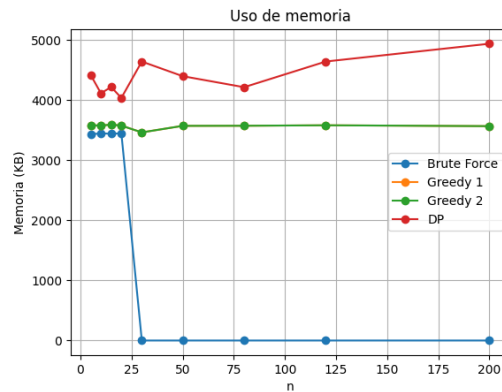


Figura 3: Uso de memoria de los algoritmos

Respecto al uso de memoria, se observa que todos los algoritmos mantienen un consumo relativamente estable para los tamaños considerados. Los algoritmos greedy requieren poca memoria adicional debido a la utilización de estructuras auxiliares simples.

El algoritmo de fuerza bruta presenta un consumo moderado asociado a las llamadas recursivas del backtracking. Por otro lado, la programación dinámica requiere almacenar estructuras bidimensionales que representan los distintos estados del problema, lo que explica su mayor consumo de memoria.

No obstante, el uso de memoria de la programación dinámica permanece dentro de límites razonables, permitiendo resolver instancias considerablemente mayores que aquellas abordables mediante fuerza bruta.

No obstante, el uso de memoria de la programación dinámica permanece dentro de límites razonables, permitiendo resolver instancias considerablemente mayores que aquellas abordables mediante fuerza bruta.

2.5. Discusión

Los resultados experimentales evidencian la existencia de un compromiso entre tiempo de ejecución, consumo de memoria y calidad de solución. La fuerza bruta garantiza soluciones óptimas, pero su costo temporal la hace impracticable para instancias medianas y grandes.

Los algoritmos greedy presentan tiempos de ejecución extremadamente bajos, aunque sacrifican calidad de solución debido a la naturaleza local de sus decisiones. Finalmente, la programación dinámica logra obtener soluciones óptimas manteniendo tiempos de ejecución razonables, convirtiéndose en la mejor alternativa para resolver el problema AniMarathon.

- **Fuerza bruta:** solución óptima, pero costo exponencial.

- **Greedy:** alta velocidad de ejecución, pero sin garantía de optimalidad.
- **Programación dinámica:** equilibrio entre eficiencia y calidad de solución.

n	Brute Force	Greedy 1	Greedy 2	DP
5	0.0088	0.0050	0.0012	1.2815
10	0.0112	0.0037	0.0010	1.1936
15	0.1247	0.0054	0.0014	1.7014
20	0.0471	0.0067	0.0019	1.1802
30	—	0.0130	0.0028	8.9891
50	—	0.0273	0.0053	14.1875
80	—	0.0212	0.0057	14.4982
120	—	0.0616	0.0090	43.8524
200	—	0.0732	0.0122	72.0294

Cuadro 2: Tiempo promedio de ejecución de los algoritmos (ms).

3. Conclusiones

Los resultados experimentales permiten concluir que la programación dinámica constituye la mejor alternativa práctica para resolver el problema AniMarathon, ya que garantiza soluciones óptimas manteniendo tiempos de ejecución razonables para tamaños moderados y grandes.

La fuerza bruta, si bien entrega soluciones óptimas, resulta impracticable para instancias grandes debido a su crecimiento exponencial. Por otro lado, las heurísticas greedy destacan por sus tiempos de ejecución extremadamente bajos, aunque presentan pérdidas de calidad en determinadas instancias.

En consecuencia, la elección del algoritmo depende del contexto del problema: si se requiere optimalidad, la programación dinámica representa la mejor opción; si se privilegia la rapidez, los métodos greedy constituyen una alternativa eficiente. Los resultados obtenidos coinciden con el análisis teórico de complejidad, validando experimentalmente el comportamiento esperado de cada enfoque.